

Evaluación de RegEx

Clase 14

IIC2223 / IIC2224

Prof. Cristian Riveros

Outline

2DFA vs DFA (anterior)

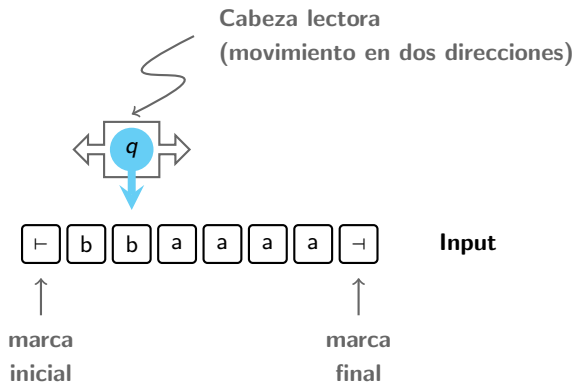
Evaluación RegEx

Outline

2DFA vs DFA (anterior)

Evaluación RegEx

Autómatas finitos deterministas en **dos direcciones**



2DFA vs lenguajes regulares

Para todo lenguaje regular L existe un 2DFA $\mathcal{A}$:

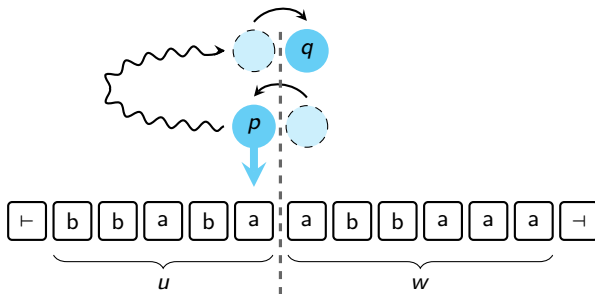
$$L = \mathcal{L}(\mathcal{A})$$

En otras palabras, $\text{DFA} \subseteq \text{2DFA}$.

¿són los 2DFA **más poderosos** que los DFA?

Demostraremos que NO!

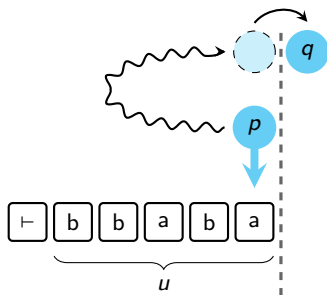
¿cuánta información puede recordar un 2DFA?



*"Cada vez que $\mathcal{A}$ cruce de w a u en el estado p ,
 $\mathcal{A}$ cruzará de regreso en el estado q ."*

este comportamiento solo depende de u y no de w !

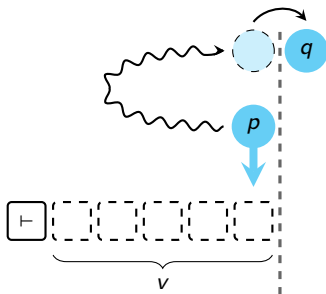
¿cuánta información puede recordar un 2DFA?



Para cada $u \in \Sigma^*$ definimos la función $T_u : Q \cup \{\bullet\} \rightarrow Q \cup \{\perp\}$ tal que:

- $T_u(p) = q$ ssi desde $(p, |u|)$ cruza en la config. $(q, |u| + 1)$.
- $T_u(p) = \perp$ ssi desde $(p, |u|)$ nunca cruza de u .
- $T_u(\bullet) = q$ ssi desde $(q_0, 0)$ cruza por 1era vez con $(q, |u| + 1)$.
- $T_u(\bullet) = \perp$ ssi desde $(q_0, 0)$ nunca cruza de u .

¿cuánta información puede recordar un 2DFA?



Suponga ahora que tenemos una palabra v tal que:

$$T_v = T_u$$

Entonces, v es **indistinguible** de u según $\mathcal{A}$

En otras palabras, $u \cdot w \in \mathcal{L}(\mathcal{A}) \Leftrightarrow v \cdot w \in \mathcal{L}(\mathcal{A})$ para todo $w \in \Sigma^*$.

Recordatorio: Teorema de Myhill-Nerode

Sea $L \subseteq \Sigma^*$ cualquier lenguaje.

Definición (Relación de Myhill-Nerode)

Una relación de equivalencia $\equiv$ en Σ^* es de **Myhill-Nerode** para L si:

1. $\equiv$ es una **congruencia por la derecha**:

$$u \equiv v \text{ entonces } u \cdot w \equiv v \cdot w \quad \forall w \in \Sigma^*$$

2. $\equiv$ **refina** L .

$$u \equiv v \text{ entonces } (u \in L \Leftrightarrow v \in L)$$

3. El número de clases de equivalencia de $\equiv$ es **finita**.

Recordatorio: Teorema de Myhill-Nerode

Definición

Dado un lenguaje $L \subseteq \Sigma^*$, se define la relación de equivalencia $\equiv_L$ como:

$$u \equiv_L v \quad \text{si, y solo si,} \quad (u \cdot w \in L \Leftrightarrow v \cdot w \in L) \quad \forall w \in \Sigma^*$$

Teorema de Myhill-Nerode

Sea $L \subseteq \Sigma^*$. Las siguientes propiedades son equivalentes:

1. L es **regular**.
2. existe una **relación de Myhill-Nerode** para L .
3. la relación $\equiv_L$ tiene una cantidad **finita** de clases de equivalencia.

¿cuánta información puede almacenar un 2DFA?

1. La relación $\equiv_T$ entre palabras en Σ^* tal que:

$$u \equiv_T v \quad \text{si, y solo si,} \quad T_u = T_v$$

es una **relación de equivalencia**.

2. $\equiv_T$ es una **congruencia por la derecha**. ($u \equiv_T v \Rightarrow \forall w. u \cdot w \equiv_T v \cdot w$)
3. $\equiv_T$ **refina** a $\mathcal{L}(\mathcal{A})$. ($u \equiv_T v \Rightarrow (u \in L \Leftrightarrow v \in L)$)
4. La relación $\equiv_T$ tiene una **cantidad finita** de clases de equivalencia.

$$T : Q \cup \{\bullet\} \rightarrow Q \cup \{\perp\}$$

¿cuántas funciones T existen?

$\equiv_{\mathcal{L}(\mathcal{A})}$ tiene una cantidad **finita** de **clases de equivalencia**.

2DFA aceptan solo lenguajes regulares

Teorema

Para todo 2DFA $\mathcal{A}$ existe un DFA $\mathcal{A}'$ tal que:

$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$$

Por lo tanto, 2DFA $\equiv$ DFA.

¿cómo construimos el DFA?

Outline

2DFA vs DFA (anterior)

Evaluación RegEx

Sobre expresiones regulares en la práctica

El lenguaje para definir expresiones regulares en la práctica se conoce como **RegEx** (o RegExp).

Las sintaxis más usadas para definir **RegEx** son:

1. POSIX (Portable Operating System Interface for uniX).
2. Perl (PCRE = Perl Compatible Regular Expressions).

Breve explicación de RegEx

	RegEx	Teoría
Carácter	<code>a</code>	a
Escape	<code>\+</code>	$-$
Cualquiera	<code>.</code>	Σ
Clase	<code>[abc]</code>	$(a + b + c)$
Clase consecutivo	<code>[a-zA-Z]</code>	$(a + \dots + z + A + \dots + Z)$
Clase exclusivo	<code>[^0-9]</code>	$(+_{a \in \Sigma - \{0, \dots, 9\}} a)$
Alternación	<code>cat dog</code>	$cat + dog$
0 o más	<code>R*</code>	R^*
1 o más	<code>R+</code>	R^+
0 o 1	<code>R?</code>	$R^?$
entre n y m	<code>R{n,m}</code>	$R^n(R + \epsilon)^{m-n}$
Backreference	<code>(R)\1</code>	$?$
...	...	...

Más información: <https://regexr.com>.

¿cómo evaluamos una expresión regular?

PROBLEMA: Evaluación de expresiones regulares

INPUT: una expresión regular R y
un documento w

OUTPUT: TRUE si, y solo si, $w \in \mathcal{L}(R)$

Tamaño del input

- $|R|$:= número de letras y operadores.
- $|w|$:= largo de documento.

... queremos un algoritmo polinomial en $|R|$ y $|w|$.

¿son ambos inputs **igual de importantes**?

Diferencia de tamaño entre expresión y documento

$$|R| \ll |w|$$

- $|R|$ puede ser del orden de **decenas** operadores ($\sim 1\text{KB}$).
- $|w|$ puede ser del orden de **miles a millones** de símbolos ($\sim 100\text{MB}$).

Análisis de tiempo diferenciado

- **Combined-complexity**: expresión y documento son parte del input.
- **Data-complexity**: solo documento es parte del input.
(tamaño de expresión es considerada una constante)

Buscamos algoritmos que sean **lineales en data-complexity** y
ojalá **polinomiales en combined-complexity**.

¿cómo evaluamos una expresión regular?

PROBLEMA: Evaluación de expresiones regulares

INPUT: una expresión regular R y
un documento w

OUTPUT: TRUE si, y solo si, $w \in \mathcal{L}(R)$

1. Convertimos R a un ϵ -NFA $\mathcal{A}_R$.
2. Verificamos si $w \in \mathcal{L}(\mathcal{A}_R)$.

¿cómo hacemos 1. y cuanto tiempo toma? ¿cómo hacemos 2.?

... dedicaremos el resto
de la clase para ver 2.

¿cómo evaluamos un autómata no-determinista?

PROBLEMA: Evaluación de NFA

INPUT: un autómata no-determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y
un documento w

OUTPUT: TRUE si, y solo si, $w \in \mathcal{L}(\mathcal{A})$

Tamaño del input

- $|w|$:= largo de documento
- $|\mathcal{A}|$:= $|Q| + |\Delta|$

Algoritmo **lineal en data-complexity** y
ojalá **polinomial en combined-complexity**.

Algoritmos de evaluación de autómatas no-deterministas

Veremos varias soluciones . . .

1. Backtracking
2. DFA
3. NFA determinización
4. NFA on-the-fly

Solución 1: Backtracking

Para un NFA $\mathcal{A} = (Q, \Sigma, \Delta, q_0, F)$ y una palabra $w = a_1 \dots a_n$.

Function eval-backtracking ($\mathcal{A}, w$)

└ **return** backtracking ($\mathcal{A}, w, q_0, 1$)

Function backtracking ($\mathcal{A}, w, p, i$)

└ **if** $i \leq n$ **then**

└ **foreach** $(p, a_i, q) \in \Delta$ **do**

└ **if** backtracking ($\mathcal{A}, w, q, i + 1$) = TRUE **then**

└ **return** TRUE

else if $q \in F$ **then**

└ **return** TRUE

return FALSE

¿cuál es el tiempo de eval-backtracking en el peor caso?

Solución 1: Backtracking

Experimento en vivo

1. Abra <https://regexr.com/> para testear expresiones regulares.
2. Pruebe la expresión regular:

$$^ (a?) \{n\} a \{n\} \$$$

sobre el input a^n donde n es un parámetro.

3. Vaya incrementando el n gradualmente.

Moraleja: *muchos motores de RegEx usan backtracking para la evaluación de expresiones regulares*

¿cuál es la ventaja de usar **backtracking**?

Solución 2: DFA

Para un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ y una palabra $w = a_1 \dots a_n$.

Function eval-DFA ($\mathcal{A}, w$)

$q := q_0$

for $i = 1$ **to** n **do**

$q := \delta(q, a_i)$

return **check** ($q \in F$)

Análisis de tiempo

■ ¿cómo hacemos $q := \delta(q, a_i)$ de manera **eficiente**?

■ ¿cuál es el tiempo de eval-DFA en el **peor caso**?

$$\mathcal{O}(|\mathcal{A}| + |w|)$$

¿para qué nos sirve **evaluar** un DFA?

Solución 3: NFA determinización

Para un NFA $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \dots a_n$.

Function eval-NFA ($\mathcal{A}, w$)

$\mathcal{A}^{\text{det}} := \text{NFAtoDFA}(\mathcal{A})$

return eval-DFA ($\mathcal{A}^{\text{det}}, w$)

Análisis de tiempo

- ¿cuál es el tiempo de eval-NFA en el **peor caso**?

$$\mathcal{O}(2^{|Q|} + |w|)$$

¿cuál es el problema con esta solución?

¿es necesario construir la determinización completa?

Recordatorio

Para un autómata no-determinista $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, se define el autómata determinista (**determinización** de $\mathcal{A}$):

$$\mathcal{A}^{\text{det}} = (2^Q, \Sigma, \delta^{\text{det}}, q_0^{\text{det}}, F^{\text{det}})$$

■ $2^Q = \{S \mid S \subseteq Q\}$ es el conjunto potencia de Q .

■ $q_0^{\text{det}} = I$.

■ $\delta^{\text{det}} : 2^Q \times \Sigma \rightarrow 2^Q$ tal que:

$$\delta^{\text{det}}(S, a) = \{q \in Q \mid \exists p \in S. (p, a, q) \in \Delta\}$$

■ $F^{\text{det}} = \{S \in 2^Q \mid S \cap F \neq \emptyset\}$.

Solución 4: NFA *on-the-fly*

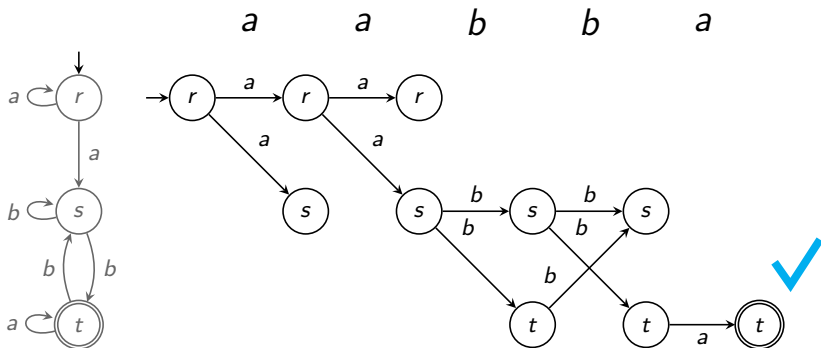
$\delta^{\text{det}} : 2^Q \times \Sigma \rightarrow 2^Q$ tal que:

$$\delta^{\text{det}}(S, a) = \{q \in Q \mid \exists p \in S. (p, a, q) \in \Delta\}$$

Estrategia on-the-fly

1. Mantenemos un conjunto S de estados actuales.
2. Por cada nueva letra a , calculamos el conjunto $\delta^{\text{det}}(S, a)$.

Solución 4: NFA *on-the-fly* (ejemplo)



Solución 4: NFA *on-the-fly*

Para un NFA $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \dots a_n$.

Function eval-NFAonthe-fly ($\mathcal{A}, w$)

```

  S := I
  for i = 1 to n do
    Sold := S
    S := ∅
    foreach p ∈ Sold do
      S := S ∪ {q | (p, ai, q) ∈ Δ}
  return check(S ∩ F ≠ ∅)
```

Análisis de tiempo

- ¿cuál es el tiempo de eval-NFAonthe-fly en el **peor caso**? $\mathcal{O}(|\mathcal{A}| \cdot |w|)$

¿es posible mejorar este algoritmo?

... es posible demostrar que no.

Resumen de técnicas de evaluación simple

Para un autómata $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \dots a_n$:

	Tiempo
Backtracking	$\mathcal{O}(\mathcal{A} ^{ w })$
DFA	$\mathcal{O}(\mathcal{A} + w)$
NFA	$\mathcal{O}(2^{ Q } + w)$
NFA on-the-fly	$\mathcal{O}(\mathcal{A} \cdot w)$

¿cuál funciona mejor **en la práctica**?

Bonus: NFA *on-the-fly* en la práctica

Para un NFA $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \dots a_n$.

Function eval-NFAonthe-fly ($\mathcal{A}, w$)

```
let  $H :=$  HashTable en  $2^Q \times \Sigma$   
 $S := I$   
for  $i = 1$  to  $n$  do  
  if  $H(S, a_i)$  is not null then  
     $S := H(S, a_i)$   
  else  
     $S_{old} := S$   
     $S := \emptyset$   
    foreach  $p \in S_{old}$  do  
       $S := S \cup \{q \mid (p, a_i, q) \in \Delta\}$   
     $H(S_{old}, a_i) := S$   
return check( $S \cap F \neq \emptyset$ )
```

¿cuál es la ventaja de este algoritmo en la práctica?

Cierre de clase

En esta clase vimos:

1. Evaluación de regex.
2. Análisis de algoritmos de evaluación de autómatas.
3. Soluciones para evaluar un NFA en la práctica.

Próxima clase: Transductores.